



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 8: Suite de Pruebas Unitarias e Integración en Capas con JUnit 5 Jupiter, Mockito y MockMvc

Resumen

Esta práctica guiada da continuidad directa a la **Práctica Guiada 7**, incorporando todos los conceptos teóricos desarrollados en el **Tema 3.7 (Pruebas de Unidad)**. Usted aprenderá a aislar y validar la lógica de negocio y los controladores REST de la plataforma de alquiler de películas **VideoRent**.

A lo largo del laboratorio, usted configurará el ecosistema de pruebas de Spring Boot utilizando **JUnit 5 Jupiter**, el motor de mocks **Mockito**, el cliente simulado **MockMvc** para la capa web, y la herramienta de reporte de cobertura **JaCoCo** a través de **Maven**.

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java 21 (o versión local instalada), Spring Boot 3.x, Maven, JUnit 5 Jupiter, Mockito 5.x, VS Code / IntelliJ IDEA.

Metas de Aprendizaje:

1. Configurar Maven con `maven-surefire-plugin` y `jacoco-maven-plugin` para la ejecución automatizada y medición de cobertura de pruebas.
2. Diseñar e implementar pruebas unitarias para servicios de negocio utilizando `@ExtendWith(MockitoExtension.class)`, `@Mock`, `@InjectMocks` y stubs declarativos (`when().thenReturn()`).
3. Validar el manejo de excepciones de negocio y aserciones de excepciones con `assertThrows()` y pruebas parametrizadas (`@ParameterizedTest`).
4. Construir pruebas de corte de controlador (*Controller Slices*) con `@WebMvcTest` y `MockMvc` para validar respuestas HTTP (200, 201, 400, 404) y payloads JSON (`jsonPath`).
5. Diagnosticar y corregir fallos comunes en entornos de prueba como `NullPointerException` por mocks no inicializados y `UnnecessaryStubbingException`.

Conceptos Clave (El 'Qué')

1. Pruebas Unitarias con JUnit 5 Jupiter

JUnit 5 está compuesto por Jupiter, Vintage y Platform. Las pruebas unitarias validan unidades mínimas de código (métodos de servicio) en completo aislamiento de la base de datos y la red.

2. Aislamiento con Mocks (Mockito)

Un *Mock* es una simulación de un objeto real. Mediante `@Mock` se crean instancias simuladas de repositorios o componentes dependientes, y con `@InjectMocks` se inyectan en la clase bajo prueba. La instrucción `when(mock.metodo()).thenReturn(valor)` define el comportamiento esperado

(*Stubbing*).

3. Pruebas de Capa Web con MockMvc

En lugar de levantar un servidor Tomcat real, `@WebMvcTest` carga únicamente la capa web de Spring Boot. `MockMvc` simula peticiones HTTP (GET, POST, PUT, DELETE) enviando JSONs y verificando los códigos de estado y la estructura de la respuesta.

4. Cobertura de Código con JaCoCo

JaCoCo (*Java Code Coverage*) analiza las líneas y ramas de bytecode ejecutadas durante el comando `mvn test`, generando reportes visuales en HTML sobre el porcentaje de cobertura del proyecto.

Parte 1: Práctica Guiada Paso a Paso

Paso 1: Configuración de Dependencias y Plugins en `pom.xml`

Asegúrese de incluir `spring-boot-starter-test` y el plugin de cobertura JaCoCo en su archivo `pom.xml`:

```
<dependencies>
  <!-- Starter para Pruebas en Spring Boot -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <!-- Plugin para ejecutar pruebas automatizadas -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
```

```

    <!-- Plugin JaCoCo para reporte de cobertura -->
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.11</version>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

```

Paso 2: Pruebas Unitarias de la Capa de Servicio (VideoServiceTest)

Cree la clase de prueba en el directorio `src/test/java/com/videorent/service/VideoServiceTest.java`. Proveeremos pruebas para el registro de películas y alquiler de títulos.

```

package com.videorent.service;

import com.videorent.exception.ResourceNotFoundException;
import com.videorent.model.Video;
import com.videorent.repository.VideoRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.mockito.InjectMocks;

```

```

import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class VideoServiceTest {

    @Mock
    private VideoRepository videoRepository;

    @InjectMocks
    private VideoService videoService;

    private Video videoEjemplo;

    @BeforeEach
    void setUp() {
        videoEjemplo = new Video();
        videoEjemplo.setId(1L);
        videoEjemplo.setTitulo("Inception");
        videoEjemplo.setDisponible(true);
        videoEjemplo.setPrecioAlquiler(3.99);
    }

    @Test
    @DisplayName("Debe registrar un nuevo video exitosamente")
    void registrarVideo_Exitoso() {
        // Arrange
        when(videoRepository.save(any(Video.class)))
            .thenReturn(videoEjemplo);

        // Act
        Video resultado = videoService.registrarVideo(videoEjemplo);

        // Assert
        assertNotNull(resultado);
        assertEquals("Inception", resultado.getTitulo());
        verify(videoRepository, times(1)).save(videoEjemplo);
    }

    @Test

```

```

@DisplayName("Debe retornar un video por su ID cuando existe")
void obtenerPorId_Existe_RetornaVideo() {
    // Arrange
    when(videoRepository.findById(1L))
        .thenReturn(Optional.of(videoEjemplo));

    // Act
    Video resultado = videoService.obtenerPorId(1L);

    // Assert
    assertNotNull(resultado);
    assertEquals(1L, resultado.getId());
    verify(videoRepository).findById(1L);
}

@Test
@DisplayName("Debe lanzar ResourceNotFoundException cuando el video no existe")
void obtenerPorId_NoExiste_LanzaExcepcion() {
    // Arrange
    when(videoRepository.findById(99L))
        .thenReturn(Optional.empty());

    // Act & Assert
    ResourceNotFoundException ex = assertThrows(
        ResourceNotFoundException.class,
        () -> videoService.obtenerPorId(99L)
    );

    assertTrue(ex.getMessage().contains("99"));
    verify(videoRepository).findById(99L);
}

@ParameterizedTest
@ValueSource(doubles = {0.0, -1.5, -10.0})
@DisplayName("Debe rechazar precios de alquiler invalidos")
void registrarVideo_PrecioInvalido_LanzaExcepcion(double precioInvalido) {
    // Arrange
    videoEjemplo.setPrecioAlquiler(precioInvalido);

    // Act & Assert
    assertThrows(IllegalArgumentException.class, () -> {
        videoService.registrarVideo(videoEjemplo);
    });

    verify(videoRepository, never()).save(any());
}

```

```
}
```

Paso 3: Pruebas de la Capa de Controladores REST (VideoControllerTest)

Cree la clase `VideoControllerTest.java` en `src/test/java/com/videorent/controller/VideoControllerTest.java` para validar las respuestas HTTP sin levantar la base de datos real:

```
package com.videorent.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.videorent.exception.ResourceNotFoundException;
import com.videorent.model.Video;
import com.videorent.service.VideoService;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(VideoController.class)
@AutoConfigureMockMvc(addFilters = false)
class VideoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @MockBean
    private VideoService videoService;
```

```

@Test
@DisplayName("GET /api/videos/1 debe retornar HTTP 200 y JSON con el video")
void obtenerVideo_Existe_Retorna200yJson() throws Exception {
    // Arrange
    Video video = new Video();
    video.setId(1L);
    video.setTitulo("The Matrix");
    video.setDisponible(true);
    video.setPrecioAlquiler(4.50);

    when(videoService.obtenerPorId(1L)).thenReturn(video);

    // Act & Assert
    mockMvc.perform(get("/api/videos/1")
        .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value(1))
        .andExpect(jsonPath("$.titulo").value("The Matrix"))
        .andExpect(jsonPath("$.disponible").value(true));
}

@Test
@DisplayName("GET /api/videos/99 debe retornar HTTP 404 Not Found")
void obtenerVideo_NoExiste_Retorna404() throws Exception {
    // Arrange
    when(videoService.obtenerPorId(99L))
        .thenThrow(new ResourceNotFoundException("Video no encontrado"));

    // Act & Assert
    mockMvc.perform(get("/api/videos/99"))
        .andExpect(status().isNotFound());
}

@Test
@DisplayName("POST /api/videos debe crear video y retornar HTTP 201")
void crearVideo_DatosValidos_Retorna201() throws Exception {
    // Arrange
    Video entrada = new Video();
    entrada.setTitulo("Interstellar");
    entrada.setPrecioAlquiler(4.99);

    Video guardado = new Video();
    guardado.setId(10L);
    guardado.setTitulo("Interstellar");
    guardado.setPrecioAlquiler(4.99);
}

```

```

        when(videoService.registrarVideo(any(Video.class)))
            .thenReturn(guardado);
    }

    // Act & Assert
    mockMvc.perform(post("/api/videos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(entrada)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").value(10))
        .andExpect(jsonPath("$.titulo").value("Interstellar"));
    }
}

```

Paso 4: Ejecución de Pruebas y Reporte de Cobertura con Maven

Ejecute la suite completa de pruebas desde su terminal en la raíz del proyecto:

```
mvn clean test
```

Al finalizar exitosamente la compilación y ejecución de las pruebas, inspeccione el reporte visual de JaCoCo generado en la siguiente ruta:

```
target/site/jacoco/index.html
```

Abra dicho archivo en su navegador web para verificar el porcentaje de cobertura de instrucciones, ramas (*branches*) y líneas de código.

Parte 2: Diagnóstico y Resolución de Errores Comunes

Escenario A: `NullPointerException` al invocar un servicio en la prueba

- **Causa:** Ha olvidado anotar la clase de prueba con `@ExtendWith(MockitoExtension.class)` o `@SpringBootTest`, por lo que las anotaciones `@Mock` e `@InjectMocks` quedan nulas.

- **Solución:** Verifique la presencia de `@ExtendWith(MockitoExtension.class)` en el encabezado de su clase de prueba unitaria.

Escenario B: `UnnecessaryStubbingException`

- **Causa:** Ha configurado una instrucción `when(mock...).thenReturn(...)` pero la prueba nunca llega a ejecutar dicho método del mock.
- **Solución:** Elimine los stubs innecesarios o configure la regla estricta de Mockito como lenient: `lenient().when(...)`.

Parte 3: Reto Autónomo (Evaluado)

Para completar esta práctica, aplique los conceptos aprendidos para extender la suite de pruebas del módulo de seguridad de **VideoRent**:

- **[] Tarea 1: Pruebas Unitarias para `TokenBlacklistService`**
Cree la clase `TokenBlacklistServiceTest` y escriba pruebas unitarias para validar los métodos `invalidarToken(String token)` y `esTokenInvalido(String token)`.
- **[] Tarea 2: Pruebas de Controlador para `AuthController`**
Cree la clase `AuthControllerTest` utilizando `@WebMvcTest(AuthController.class)` para simular peticiones `POST /api/auth/login` con credenciales válidas e inválidas, verificando respuestas HTTP 200 OK y 401 Unauthorized.
- **[] Tarea 3: Verificación de Cobertura JaCoCo**
Ejecute `mvn test` y confirme que la cobertura general en el paquete de servicios alcance al menos un **80%**.

Lista de Chequeo Final

Antes de dar por concluida la práctica, asegúrese de haber verificado los siguientes puntos:

- [] Todas las pruebas en `VideoServiceTest` y `VideoControllerTest` se ejecutan sin fallos (`BUILD SUCCESS`).
- [] Las aserciones `assertThrows` capturan correctamente las excepciones `ResourceNotFoundException`.
- [] `MockMvc` valida correctamente las estructuras JSON devueltas mediante `jsonPath`.
- [] El reporte JaCoCo fue generado correctamente en `target/site/jacoco/index.html`.